

Decorate Me API 架構與串接流程規格

臉部分析獨立 API、前端/iOS 共用、非同步機制、資料包格式與各端串接

版本日期：2026-06-13

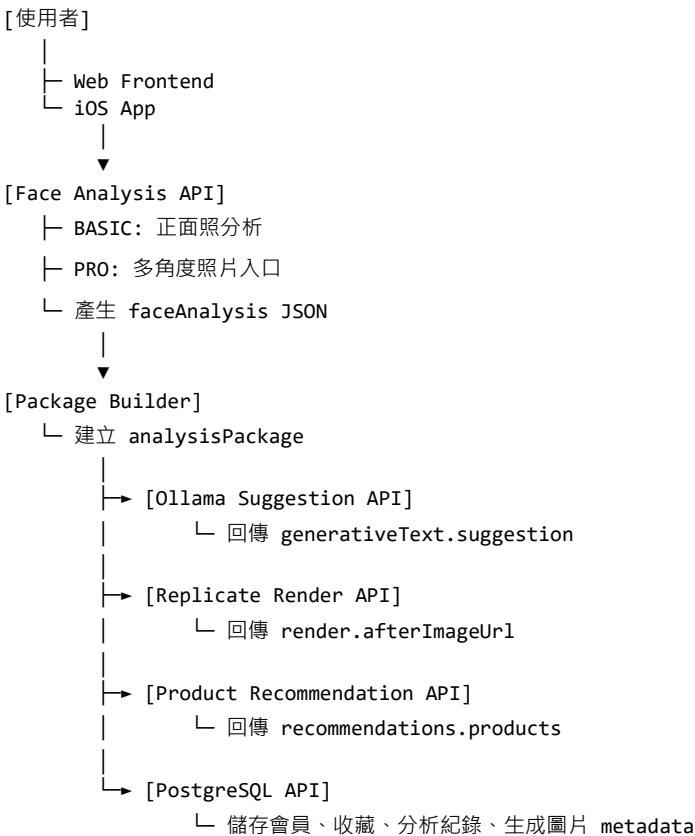
核心決策

臉部分析模組要獨立成 API，不綁死網頁前端。網頁與 iOS 都只呼叫同一套 Face Analysis API。為了避免多人同時使用時爆掉，分析流程應從同步 API 逐步升級成 Job Queue 非同步流程。所有跨模組資料交換以 analysisPackage 為核心格式。

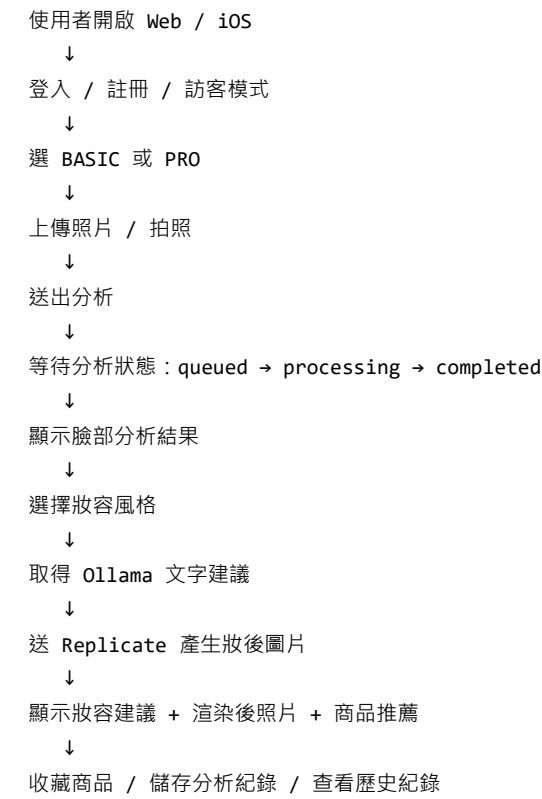
1. 為什麼臉部分析模組要獨立 API

- 未來網頁版與 iOS App 都會使用同一個臉部分析能力，不能把分析邏輯寫死在前端或只服務網頁。
- 臉部分析、Ollama 文字建議、Replicate 渲染、PostgreSQL 資料庫都應該是獨立服務，透過 API 串接。
- 前端與 iOS 只需要知道 API contract，不需要知道後端內部用 MediaPipe、InsightFace 或未來模型。
- 獨立 API 可以單獨部署、測試、擴充，也比較容易做非同步排隊與錯誤處理。

2. 系統總流程圖



3. 使用者流程圖



4. 臉部分析 API 設計

短期可以保留目前同步 API，但正式串接 Web 與 iOS 時，建議新增 v1 API 命名與非同步任務 API。

Endpoint	Method	用途	呼叫者	輸出
/v1/face/analyze/basic	POST	BASIC 正面照同步分析	Web / iOS	faceAnalysis JSON 或 analysisPackage
/v1/face/analyze/pro	POST	PRO 多角度同步分析入口	Web / iOS	PRO analysisPackage
/v1/face/jobs/basic	POST	建立 BASIC 非同步分析任務	Web / iOS	jobId
/v1/face/jobs/pro	POST	建立 PRO 非同步分析任務	Web / iOS	jobId
/v1/face/jobs/{jobId}	GET	查詢任務狀態	Web / iOS	queued/processing/completed/failed
/v1/face/jobs/{jobId}/result	GET	取得分析結果	Web / iOS	analysisPackage

4.1 BASIC 同步 API

```
POST /v1/face/analyze/basic
Content-Type: multipart/form-data

Request:
file: image
client: "web" | "ios"
```

```
userId: optional

Response:
{
  "status": "completed",
  "analysisPackage": {
    "id": "AN-xxx",
    "mode": "BASIC",
    "faceAnalysis": {...}
  }
}
```

4.2 PRO 同步 API

```
POST /v1/face/analyze/pro
Content-Type: multipart/form-data
```

```
Request:
front: image
left45: image optional
right45: image optional
side: image optional
client: "web" | "ios"
userId: optional

Response:
{
  "status": "completed",
  "analysisPackage": {
    "id": "AN-xxx",
    "mode": "PRO",
    "faceAnalysis": {...},
    "proAnalysis": {
      "status": "reserved",
      "notes": "多角度精細分類待實作"
    }
  }
}
```

5. 非同步機制設計

為什麼需要非同步

目前同步 API 在使用者少時可以運作，但如果多人同時上傳照片，臉部分析、Ollama、Replicate 都可能耗時。同步流程會讓請求卡住，甚至造成後端 worker 被占滿。非同步 job queue 可以把請求先排隊，前端輪詢狀態，避免整個服務爆掉。

階段	做法	適合時機
第一階段	保留同步 /analyze，前端加 loading 與 timeout	目前開發與展示
第二階段	新增 jobs API，先用 FastAPI BackgroundTasks + 記憶體狀態表	小型 demo、多人測試
第三階段	改用 Redis + Celery/RQ，job 狀態存在 PostgreSQL	正式部署、多人使用

第四階段	圖片上傳改 object storage · API 只傳 imageUrl/url	使用者量變大、圖片變多
------	--	-------------

5.1 非同步流程圖

```
Web / iOS
├─ POST /v1/face/jobs/basic
│   └─ Response: { jobId: "JOB-xxx", status: "queued" }
├─ GET /v1/face/jobs/JOB-xxx
│   └─ Response: queued / processing / completed / failed
└─ GET /v1/face/jobs/JOB-xxx/result
    └─ Response: analysisPackage
```

5.2 Job 狀態格式

```
{
  "jobId": "JOB-xxx",
  "analysisPackageId": "AN-xxx",
  "status": "queued | processing | completed | failed",
  "progress": 0,
  "stage": "upload | face_analysis | package | suggestion | render | done",
  "createdAt": "...",
  "startedAt": "...",
  "completedAt": "...",
  "error": null
}
```

6. analysisPackage 資料包格式

analysisPackage 是所有端交換資料的核心。臉部分析端、Ollama、Replicate、商品、資料庫都應該讀寫這包資料的不同區塊。

```
{
  "id": "AN-xxx",
  "schemaVersion": "2026-06-v1",
  "mode": "BASIC",
  "client": "web | ios",
  "userId": null,
  "status": "completed",
  "createdAt": "...",
  "updatedAt": "...",

  "images": {
    "front": {
      "originalName": "photo.jpg",
      "originalType": "image/jpeg",
      "originalSize": 1234567,
      "compressedImageUrl": null,
      "compressedDataUrl": "base64-for-package",
      "compressedWidth": 1024,
      "compressedHeight": 1024
    }
  }
},
```

```

"faceAnalysis": {
  "version": "BASIC",
  "faceShape": "oval",
  "browShape": "curved",
  "eyeShape": "peach_blossom",
  "noseFront": "standard",
  "lipShape": "smile",
  "skinTone": {
    "season": "spring",
    "level": "白晳自然色",
    "lab": {"L": 70, "a": 10, "b": 14}
  },
  "raw": {
    "臉型": "鵝蛋臉",
    "眉型": "彎月眉",
    "眼型": "桃花眼",
    "鼻型": "標準鼻",
    "嘴型": "微笑唇",
    "膚色": {}
  }
},

"generativeText": {
  "status": "pending | completed | failed",
  "provider": "ollama",
  "model": "gemma3 / llava",
  "suggestion": null,
  "error": null
},

"render": {
  "status": "pending | completed | failed",
  "provider": "replicate",
  "replicateTempUrl": null,
  "afterImageUrl": null,
  "savedImageId": null,
  "error": null
},

"recommendations": {
  "products": [],
  "tips": [],
  "ads": []
}
}

```

7. 每一端接誰的 API、輸入什麼、輸出什麼

端點/組別	接誰的 API	輸入	輸出	輸出方式
Web 前端	Face API、Ollama API、Render API、Product API、DB API	照片、使用者操作、analysisPackage	畫面顯示、收藏、歷史紀錄	HTTP API
iOS App	同 Web，接 Face API 與其他 API	照片、使用者操作	App 畫面、analysisPackage	HTTP API

Face Analysis API	可選：不直接接其他端；或後續接 Package Builder	front/left45/right45/side 圖片	faceAnalysis JSON / analysisPackage	HTTP API / JSON
Ollama 文字建議端	接收 Package Builder 或前端傳來的 analysisPackage	faceAnalysis、style、compressedImage	generativeText.suggestion	HTTP API JSON
Replicate 渲染端	接收 analysisPackage	compressedImage、faceAnalysis、generativeText、style	afterImageUrl、savedImageId	HTTP API JSON
商品推薦端	接收 analysisPackage 或 faceAnalysis + style	faceAnalysis、skinTone、style、suggestion	products[]	HTTP API JSON
PostgreSQL API	接收前端或後端送來的紀錄	user、favorite、analysisPackage、generatedImage metadata	資料庫紀錄 id	HTTP API JSON

8. API Contract 最低要求

- 每個服務都必須有 GET /health。
- 每個 response 都必須有 status。
- 錯誤時必須回 error.message，不可只讓前端收到空白或 HTML error page。
- 圖片欄位必須明確：File、base64、DataURL、imageUrl 不可混用不說明。
- Replicate 端不可只回 replicateTempUrl，必須回保存後的 afterImageUrl。
- PostgreSQL 不直接暴露給前端，必須透過後端 API。
- Ollama 不直接暴露 11434 給前端，必須由組員包 /suggest API。

9. 推薦的後續開發順序

1. 先把 Face Analysis API 定成 /v1 命名，讓 Web 與 iOS 可共用。
2. 新增 analysisPackage v1 格式，前端和後端都照同一份 schema。
3. 保留同步 API，但新增非同步 jobs API 草案。
4. 短期先讓前端打同步 API，並用 fallback 防止其他端未開造成 demo 中斷。
5. Ollama、Replicate、商品、DB 組員各自提供 /health 與 mock endpoint。
6. 完成第一輪整合後，再把同步流程改成 job queue。
7. 最後再補 PRO 多角度精細分類，不要先卡在 PRO。

10. 會議中可以直接定下來的規則

規則	原因
Face Analysis API 必須獨立	Web 與 iOS 都會共用
analysisPackage 是唯一資料交換核心	避免各端自定格式造成整合混亂
每個服務都要有 /health	串接會議時先確認服務是否活著
每天固定短串接	大家都接前端，不能等最後一天才測
圖片 URL 必須保存成永久 URL	Replicate 臨時 URL 會失效
先 BASIC，後 PRO	避免精細分類拖慢主流程
先 mock，後真模型	前端流程可以先接起來，減少互相等待